

Komunikacja i synchronizacja procesów

Sygnały, współbieżność, sekcja krytyczna i zakleszczenia

Bernadetta Bartosik

Komunikacja i synchronizacja procesów stanowią fundament działania współczesnych systemów operacyjnych oraz aplikacji wielowątkowych. W środowisku, w którym wiele procesów i wątków działa jednocześnie, konieczne jest zapewnienie mechanizmów umożliwiających ich bezpieczną współpracę.

Brak odpowiedniej synchronizacji prowadzi do błędów logicznych, niespójności danych oraz zachowań trudnych do przewidzenia i debugowania. Dlatego systemy operacyjne oferują różne mechanizmy umożliwiające kontrolę dostępu do zasobów oraz komunikację między procesami.

Proces jest niezależną jednostką wykonania programu, posiadającą własną przestrzeń adresową, co zapewnia izolację od innych procesów. Dzięki temu błędy w jednym procesie nie wpływają bezpośrednio na inne.

Wątki są lżejszymi jednostkami wykonania działającymi w obrębie jednego procesu. Współdzielą one pamięć, co znacząco przyspiesza komunikację, ale jednocześnie zwiększa ryzyko wystąpienia problemów współbieżności.

Z punktu widzenia programisty oznacza to konieczność większej ostrożności przy operacjach na wspólnych danych.

Współbieżność a równoległość

Współbieżność odnosi się do zdolności systemu do obsługi wielu zadań w tym samym czasie logicznym poprzez przełączanie kontekstu. Równoległość natomiast oznacza rzeczywiste jednoczesne wykonywanie wielu zadań na różnych rdzeniach procesora.

W praktyce współbieżność może występować nawet na systemach jednordzeniowych, podczas gdy równoległość wymaga sprzętowego wsparcia.

Problem współdzielenia zasobów

Współdzielenie zasobów takich jak pamięć, pliki czy urządzenia wejścia/wyjścia prowadzi do potencjalnych konfliktów. Jeśli dwa procesy jednocześnie modyfikują ten sam zasób, wynik operacji może być niepoprawny.

Klasycznym przykładem jest inkrementacja zmiennej przez wiele wątków, gdzie brak synchronizacji prowadzi do utraty części operacji.

Komunikacja między procesami (IPC)

Mechanizmy IPC umożliwiają procesom wymianę danych oraz synchronizację działań. W zależności od potrzeb stosuje się różne rozwiązania, które różnią się wydajnością, poziomem abstrakcji oraz zastosowaniem.

Wybór odpowiedniego mechanizmu IPC zależy od charakteru aplikacji oraz wymagań dotyczących wydajności i bezpieczeństwa.

Potoki są jednym z najprostszych mechanizmów komunikacji między procesami. Umożliwiają przesyłanie danych w sposób strumieniowy, zazwyczaj między procesem rodzicem a potomkiem.

Potok można sobie wyobrazić jako jednokierunkowy tunel w pamięci RAM, zarządzany przez jądro systemu operacyjnego. Dane są do niego wpisane z jednego końca i odczytywane z drugiego zgodnie z zasadą FIFO (First-In, First-Out).

- **Potoki nienazwane**

Istnieją tylko w pamięci i żyją tak długo, jak procesy, które je stworzyły. Zazwyczaj służą do komunikacji między procesem rodzica a procesem potomnym (utworzonym przez `fork()`).

- **Potoki nazwane**

Są widoczne w systemie plików jako specjalne pliki. Pozwalają na komunikację między zupełnie niespokrewnionymi procesami, które po prostu "znają" ścieżkę do tego pliku.

❶ Jednokierunkowość (Simplex)

Standardowy potok jest jednokierunkowy. Jeśli proces A i proces B chcą ze sobą swobodnie rozmawiać "w obie strony", muszą otworzyć dwa oddzielne potoki.

❷ Brak struktury danych

Potok przesyła "surowy" strumień bajtów. Nie wie, gdzie kończy się jedna wiadomość, a zaczyna druga. To programista musi zdefiniować protokół (np. czytać po 1024 bajty lub szukać znaku nowej linii \n), aby poprawnie zinterpretować dane.

❸ Ograniczony rozmiar bufora

Potoki mają określoną pojemność (zależną od systemu, np. 64 KB). Jeśli proces produkujący dane jest znacznie szybszy niż odbierający, może dojść do zablokowania nadawcy do czasu, aż odbiorca zwolni miejsce w "rurze".

Kolejki komunikatów pozwalają na przesyłanie uporządkowanych wiadomości między procesami. Każdy komunikat może mieć określony typ oraz priorytet, co umożliwia bardziej zaawansowane scenariusze komunikacji.

Są one szczególnie przydatne w systemach wymagających asynchronicznej wymiany danych.

- Asynchroniczność i "Odklejanie"(Decoupling)

To najważniejsza zaleta. Nadawca może wysłać wiadomość i natychmiast przejść do kolejnych zadań, nie czekając, aż odbiorca ją przetworzy. Jeśli odbiorca jest zajęty lub tymczasowo wyłączony, wiadomość bezpiecznie czeka w kolejce wewnątrz jądra systemu.

- Brak konieczności pokrewieństwa

W przeciwieństwie do potoków nienazwanych, kolejki komunikatów mogą być używane przez zupełnie obce procesy. Wystarczy, że znają one wspólny identyfikator (klucz) kolejki.

- Trwałość (Persistence)

Wiadomości w kolejce istnieją tak długo, aż zostaną jawnie odebrane lub system zostanie zrestartowany (zależnie od implementacji). Jeśli proces odbiorcy ulegnie awarii, dane przesłane przez nadawcę nie przepadają.

Pamięć współdzielona jest najszybszym mechanizmem IPC, ponieważ eliminuje konieczność kopiowania danych między procesami. Procesy uzyskują dostęp do tego samego obszaru pamięci.

- Brak kopiowania: Gdy Proces A zapisze wartość w pamięci współdzielonej, jest ona natychmiast widoczna dla Procesu B, ponieważ obaj patrzą na te same komórki fizycznej pamięci.
- Szybkość: Jest to rozwiązanie niemal tak szybkie, jak dostęp do pamięci lokalnej wewnątrz jednego procesu.

Jednak brak wbudowanej synchronizacji oznacza, że programista musi samodzielnie zadbać o ochronę danych przed jednoczesnym dostępem.

- ❶ **Brak wbudowanej kontroli** W potokach czy kolejkach jądro systemu dba o to, by dwa procesy nie "zderzyły się" podczas zapisu. W pamięci współdzielonej nie ma żadnego mechanizmu kolejkowania. Jeśli dwa procesy spróbują zapisać dane w tym samym miejscu w tym samym mikrosekundzie, dojdzie do uszkodzenia danych (race condition).
- ❷ **Konieczność zewnętrznej synchronizacji** Programista korzystający z pamięci współdzielonej musi niemal zawsze użyć dodatkowego mechanizmu do zarządzania dostępem. Najczęściej są to:
 - Semaforey: Do sygnalizowania, czy dany blok pamięci jest obecnie zajęty, czy wolny.
 - Muteksy (w pamięci współdzielonej): Do blokowania dostępu do struktur danych.

Gniazda (sockets)

Gniazda umożliwiają komunikację zarówno lokalną, jak i sieciową. Są podstawą działania aplikacji klient-serwer i systemów rozproszonych. Ich uniwersalność sprawia, że są szeroko stosowane w aplikacjach sieciowych. Kluczowe komponenty adresu gniazda:

- Adres IP: Identyfikuje konkretną maszynę w sieci.
- Numer portu: Identyfikuje konkretny proces (aplikację) na tej maszynie (np. port 80 dla HTTP, 443 dla HTTPS).
- Protokół: Określa zasady wymiany danych (najczęściej TCP lub UDP).

- ❶ **Gniazda Strumieniowe (Stream Sockets – TCP)**
Działają w oparciu o protokół TCP. Są zorientowane na połączenie, co oznacza, że przed wysłaniem danych musi zostać nawiązane połączenie trójstopniowe.
 - Zaleta: Gwarancja dostarczenia danych w poprawnej kolejności i bez błędów.
 - Zastosowanie: Przeglądarki WWW, przesyłanie plików (FTP), bazy danych.
- ❷ **Gniazda Datagramowe (Datagram Sockets – UDP)**
Działają w oparciu o protokół UDP. Są bezpołączeniowe – proces po prostu wysyła paczkę danych w stronę odbiorcy.
 - Zaleta: Minimalne opóźnienia, brak narzutu na sprawdzanie błędów.
 - Zastosowanie: Streaming wideo, gry online, rozmowy głosowe (VoIP).
- ❸ **Gniazda Lokalne (Unix Domain Sockets)** Specjalny rodzaj gniazd używany wyłącznie do komunikacji wewnątrz tego samego systemu operacyjnego. Zamiast adresów IP używają ścieżek w systemie plików. Są znacznie szybsze niż gniazda sieciowe, bo pomijają całą warstwę stosu sieciowego.

Jak jądro komunikuje się z procesem?

- Natura sygnału: Jest to programowe przerwanie, które wymusza na procesie nagłą zmianę toku wykonania.
- Kto wysyła?
 - Jądro systemu: Np. gdy proces spróbuje odwołać się do nielegalnej pamięci (SIGSEGV).
 - Użytkownik: Przez skróty klawiszowe (Ctrl+C) lub polecenie kill.
 - Inny proces: Jeśli ma odpowiednie uprawnienia.

Tabela: Podstawowe sygnały w systemach UNIX-owych

Sygnał	Opis	Działanie domyślne
SIGINT	Przerwanie z terminala	Zakończenie (można przechwycić)
SIGKILL	Bezwzględne zabicie	Natychmiastowy koniec (niezatrzymywalny)
SIGALRM	Budzik (timer)	Zakończenie
SIGCHLD	Zmiana stanu potomka	Ignorowanie

Zarządzanie asynchronicznością

- Struktura obsługi: Proces po otrzymaniu sygnału może zareagować zgodnie z zaprogramowaną logiką:
 - Maska sygnałów: Proces może tymczasowo "zablokować" określone sygnały, aby nie przerywały one krytycznych obliczeń.
 - Funkcja obsługi (Handler): Specjalna funkcja, do której system "skacze" w momencie nadejścia sygnału. Po jej zakończeniu program wraca do miejsca, w którym został przerwany.
- Dlaczego to ważne? Pozwala na "bezpieczne wyłączenie" aplikacji (np. zamknięcie plików przed wyjściem po naciśnięciu Ctrl+C).

Wyścig (Race Condition)

Sytuacja, w której wynik końcowy zależy od tego, który proces był szybszy (kolejności wykonania).

Przykład inkrementacji:

Operacja $x = x + 1$ na poziomie procesora składa się z trzech kroków:

- Pobierz x do rejestru.
- Dodaj 1.
- Zapisz x z powrotem do pamięci.

Problem: Jeśli dwa wątki zrobią krok 1 jednocześnie, oba pobiorą tę samą wartość (np. 10). Po dodaniu 1 oba zapiszą 11. Wynik powinien wynosić 12, a wynosi 11 – jedna operacja została "zgubiona".

Algorytmy i sprzęt

- Podejście programowe: Np. Algorytm Petersona – wykorzystuje zmienne flagowe i "kolejkę", aby wymusić wzajemne wykluczenie bez wsparcia sprzętowego.
- Wsparcie sprzętowe: Procesory oferują instrukcje atomowe, np. TSL (Test-and-Set Lock).
 - Odczytuje wartość zmiennej i ustawia ją na 1 w jednej, niepodzielnej operacji.
 - Gwarantuje to, że żaden inny proces nie "wciśnie się" pomiędzy odczyt a zapis blokady.
- Czekanie aktywne (Spinlock): Proces ciągle sprawdza warunek wejścia w pętli while. Jest szybkie, ale marnuje czas procesora.

Semafor S to chroniona zmienna całkowita, do której dostęp mają wyłącznie dwie atomowe operacje systemowe: P (wait) oraz V (signal). Implementacja wewnątrz jądra:

- Semafor nie jest zwykłym licznikiem, posiada powiązaną kolejkę procesów zablokowanych.
- Gdy proces wykonuje $P(S)$ i $S \leq 0$, jądro zmienia stan procesu na WAITING, zawiesza jego wykonanie i umieszcza w kolejce semafora.
- Operacja $V(S)$ nie tylko zwiększa licznik, ale sprawdza kolejkę: jeśli nie jest pusta, jeden z procesów jest przenoszony do stanu READY.

Atomowość: System gwarantuje, że operacje na semaforze nie zostaną przerwane przez planistę (użycie instrukcji procesora typu Test-and-Set lub blokowanie przerwania na poziomie jądra).

Muteks to uproszczona forma semafora o stanie binarnym (Locked/Unlocked), zaprojektowana specjalnie do ochrony sekcji krytycznych. Muteks "pamięta", który wątek go zablokował. Tylko ten sam wątek może go odblokować. Zapobiega to błędom logicznym, gdzie jeden proces zwalnia blokadę należącą do innego.

Optymalizacja wydajności – Futex (Fast Userspace Mutex):

- Współczesne systemy (np. Linux) starają się unikać kosztownego przełączania do trybu jądra.
- Jeśli nie ma rywalizacji o zasób (zasób jest wolny), operacja lock odbywa się całkowicie w przestrzeni użytkownika.
- Przejście do jądra następuje tylko wtedy, gdy wymagane jest uśpienie wątku z powodu zajętości zasobu.

Muteksy rekurencyjne: Pozwalają temu samemu wątkowi na wielokrotne zablokowanie tego samego muteksu bez wywołania zakleszczenia (wymagają wtedy takiej samej liczby odblokowań).

Muteksy vs Semafor

Cecha	Muteks (Mutex)	Semafor
Właściciel	Tylko proces, który go zamknął, może go otworzyć.	Dowolny proces może wysłać sygnał zwolnienia.
Zastosowanie	Ścisłe wzajemne wykluczenie (sekcja krytyczna).	Synchronizacja kolejności zadań lub zasoby licznikowe.
Typ	Mechanizm blokujący (Locking).	Mechanizm sygnalizacyjny (Signaling).

Zakleszczenia (Deadlock)

Zakleszczenie to stan, w którym każdy proces z pewnego zbioru oczekuje na zdarzenie (zwolnienie zasobu), które może zostać wywołane jedynie przez inny proces z tego samego zbioru. Wynik: procesy trwają w nieskończonym oczekiwaniu. Aby doszło do zakleszczenia, muszą wystąpić jednocześnie cztery czynniki (Warunki Coffmana):

- Wzajemne wykluczenie (Mutual Exclusion): Zasób może być używany tylko przez jeden proces naraz (jest niepodzielny).
- Przetrzymywanie i oczekiwanie (Hold and Wait): Proces trzyma co najmniej jeden zasób i jednocześnie żąda kolejnych, które są zajęte przez inne procesy.
- Brak wywłaszczania (No Preemption): Zasób nie może zostać odebrany procesowi siłą; musi on zostać zwolniony dobrowolnie.
- Czekanie cykliczne (Circular Wait): Istnieje zamknięty łańcuch procesów $\{P_0, P_1, \dots, P_n\}$, gdzie P_0 czeka na zasób P_1 , P_1 na P_2 , a P_n na zasób P_0 .

Strategie radzenia sobie z deadlockiem:

- Zapobieganie: Projektowanie systemu tak, aby wyeliminować jeden z 4 warunków.
- Unikanie: Dynamiczne sprawdzanie bezpieczeństwa przy każdym żądaniu (np. Algorytm Bankiera).
- Wykrywanie i naprawa: Pozwalanie na zakleszczenie, a następnie restartowanie procesów lub odbieranie zasobów.
- Metoda Strusia: Ignorowanie problemu (zakładanie, że występuje tak rzadko, że nie warto tracić mocy obliczeniowej na jego kontrolę – stosowane w wielu systemach operacyjnych).

Pamiętaj: Jeśli choć jeden z warunków Coffmana nie zostanie spełniony, zakleszczenie jest niemożliwe.

- Problem: Sytuacja, w której proces o niskim priorytecie trzyma zasób (np. muteks), którego potrzebuje proces o wysokim priorytecie.
- Efekt: Proces o wysokim priorytecie zostaje zablokowany, a proces o średnim priorytecie (który nie potrzebuje zasobu) przejmuje procesor, spowalniając oba pozostałe zadania.
- Historyczny przykład: Problem z misją Mars Pathfinder (1997), gdzie inwersja priorytetów powodowała ciągłe restarty komputera pokładowego.

Walka z inwersją priorytetów

- Dziedziczenie priorytetu (Priority Inheritance): Proces o niskim priorytecie czasowo "otrzymuje" priorytet procesu wysokiego, który na niego czeka. Pozwala to na szybkie zakończenie sekcji krytycznej i zwolnienie zasobu.
- Pułap priorytetu (Priority Ceiling): Każdemu zasobowi przypisuje się priorytet równy najwyższemu priorytetowi procesu, który może go użyć.
- Rezultat: Minimalizacja czasu blokowania procesów krytycznych dla działania systemu.

Klasyczny problem: Producent i Konsument

Problem ograniczonego bufora

Scenariusz: Producent generuje dane do bufora, a Konsument je pobiera.

Wymagania synchronizacji:

- Producent nie może dodać danych do pełnego bufora.
- Konsument nie może pobrać danych z pustego bufora.
- Dostęp do bufora musi być chroniony przed jednoczesną modyfikacją (sekcja krytyczna).

Rozwiązanie: Zazwyczaj stosuje się trzy semafore: empty (liczący wolne miejsca), full (liczący zajęte miejsca) oraz binarny muteks do ochrony samego bufora.